

XCS221 Assignment 4 — Controlling Mountain Car

Due Sunday, April 26 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at <http://xcs221-scpd.slack.com/>
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset $\text{\LaTeX}$ submission. If you wish to typeset your submission and are new to $\text{\LaTeX}$, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Also note that your answers should be in order and clearly and correctly labeled to receive credit. Be sure to submit your final answers as a PDF and **tag all pages correctly when submitting to Gradescope**.

Coding Submission: Some questions in this assignment require a coding response. For these questions, you should submit only the `src/submission.py` file in the online student portal. For further details, see Writing Code and Running the Autograder below.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. More information regarding the Stanford honor code can be found at <https://communitystandards.stanford.edu/policies-and-guidance/honor-code>.

Writing Code and Running the Autograder

All your code should be entered into `src/submission.py`. When editing `src/submission.py`, please only make changes between the lines containing `### START_CODE_HERE ###` and `### END_CODE_HERE ###`. Do not make changes to files other than `src/submission.py`.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- **basic:** These tests are provided to make sure that your inputs and outputs are on the right track, and that the hidden evaluation tests will be able to execute.
- **hidden:** These unit tests are the evaluated elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests. These evaluative hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the [Anaconda Setup for XCS Courses](#) to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Hidden Tests

All hidden tests rely on files that are not provided to students. Therefore, the tests can only be run remotely. When a hidden test like `1a-1-hidden` is executed locally, it will produce the following result:

```
----- START 1a-1-hidden: Test multiple instances of the same word in a sentence.
----- END 1a-1-hidden [took 0:00:00.011989 (max allowed 1 seconds), ???/3 points] (hidden test ungraded)
```

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

```
----- START 1a-0-basic: Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic: Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None ← This error caused the test to fail.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a") ← In this case, start your debugging
                                           in line 23 of grader.py.
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

Remote Execution

Basic and hidden tests are treated the same by the remote autograder. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.

1a-0-basic) Basic test case. (0.0/2.0)

```
<class 'AssertionError': {'a': 2, 'b': 1} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

Just like in the local autograder, this error caused the test to fail.

Just like in the local autograder, start your debugging in line 23 of grader.py.

1a-1-hidden) Test multiple instances of the same word in a sentence. (0.0/3.0)

```
<class 'AssertionError': {'a': 23, 'ab': 22, 'aa': 24, 'c': 16, 'b': 15} != None
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
File "/autograder/source/grader.py", line 31, in test_1
    self.compare_with_solution_or_wait(submission, 'extractWordFeatures', lambda f: f(sentence))
File "/autograder/source/graderUtil.py", line 183, in compare_with_solution_or_wait
    self.assertEqual(ans1, ans2)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

This error caused the test to fail.

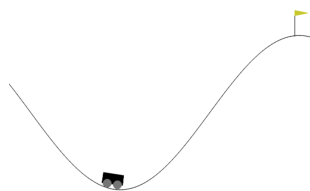
Start your debugging in line 31 of grader.py.

Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.

1a-0-basic) Basic test case. (2.0/2.0)

1a-1-hidden) Test multiple instances of the same word in a sentence. (3.0/3.0)

Introduction



Markov decision processes (MDPs) can be used to model situations with uncertainty (which is most of the time in the real world). In this assignment, you will implement algorithms to find the optimal policy, even you know the transitions and rewards (value iteration) and when you don't (reinforcement learning). You will use these algorithms on Mountain Car, a classic control environment where the goal is to control a car to go up a mountain.

Figure 1: Mountain Car

1. Value Iteration

In this problem, you will perform value iteration updates manually on a basic game to build your intuitions about solving MDPs. The set of possible states in this game is $\text{States} = \{-2, -1, 0, +1, +2\}$ and the set of possible actions is $\text{Actions}(s) = \{a_1, a_2\}$ for all states that are not end states. The starting state is 0 and there are two end states, -2 and $+2$. Recall that the transition function $T : \text{States} \times \text{Actions} \rightarrow \Delta(\text{States})$ encodes the probability of transitioning to a next state s' after being in state s and taking action a as $T(s'|s, a)$. In this MDP, the transition dynamics are given as follows:

$\forall i \in \{-1, 0, 1\} \subseteq \text{States}$,

- $T(i-1|i, a_1) = 0.8$ and $T(i+1|i, a_1) = 0.2$
- $T(i-1|i, a_2) = 0.7$ and $T(i+1|i, a_2) = 0.3$

Think of this MDP as a chain formed by states $\{-2, -1, 0, +1, +2\}$. In words, action a_1 has a 80% chance of moving the agent backwards in the chain and a 20% chance of moving the agent forward. Similarly, action a_2 has a 70% of sending the agent backwards and a 30% chance of moving the agent forward. We will use a discount factor $\gamma = 1$. The reward function for this MDP is

$$\text{Reward}(s, a, s') = \begin{cases} 10 & \text{if } s' = -2, \\ 50 & \text{if } s' = +2, \\ -5 & \text{otherwise.} \end{cases}$$

(a) [2 points (Written)]

What is the value of $V_i^*(s)$ for each $s \in \text{States}$ after each iteration $i = \{1, 2\}$ of value iteration? Recall that $\forall s \in \text{States}$, $V_0^*(s) = 0$ and, for any i , end state s_{end} , we have $V_i^*(s_{\text{end}}) = 0$.

What we expect: The $V_i^*(s)$ of all 5 states after each iteration. In total, 10 values should be reported.

The Bellman optimality update with $\gamma = 1$ is

$$V_i^*(s) = \max_a \sum_{s'} T(s' | s, a) [R(s, a, s') + V_{i-1}^*(s')].$$

The end states -2 and $+2$ satisfy $V_i^*(\pm 2) = 0$ for all i .

Iteration 1 (using $V_0^*(s) = 0$ for all s):

- $s = -1$: a_1 gives $0.8(10) + 0.2(-5) = 7$; a_2 gives $0.7(10) + 0.3(-5) = 5.5$. $\Rightarrow V_1^*(-1) = 7$.
- $s = 0$: both actions yield -5 (every successor has reward -5 and $V_0^* = 0$). $\Rightarrow V_1^*(0) = -5$.
- $s = +1$: a_1 gives $0.8(-5) + 0.2(50) = 6$; a_2 gives $0.7(-5) + 0.3(50) = 11.5$. $\Rightarrow V_1^*(+1) = 11.5$.

Iteration 2 (using V_1^* above):

- $s = -1$: a_1 gives $0.8(10+0)+0.2(-5+(-5)) = 6$; a_2 gives $0.7(10+0)+0.3(-5+(-5)) = 4$. $\Rightarrow V_2^*(-1) = 6$.
- $s = 0$: a_1 gives $0.8(-5+7) + 0.2(-5+11.5) = 2.9$; a_2 gives $0.7(-5+7) + 0.3(-5+11.5) = 3.35$. $\Rightarrow V_2^*(0) = 3.35$.
- $s = +1$: a_1 gives $0.8(-5+(-5))+0.2(50+0) = 2$; a_2 gives $0.7(-5+(-5))+0.3(50+0) = 8$. $\Rightarrow V_2^*(+1) = 8$.

Summary of all 10 values:

s	-2	-1	0	$+1$	$+2$
$V_1^*(s)$	0	7	-5	11.5	0
$V_2^*(s)$	0	6	3.35	8	0

(b) [1 point (Written)]

Using $V_2^*(\cdot)$, what is the corresponding optimal policy π^* for all non-end states?

What we expect: Optimal policy $\pi^*(s)$ for each non-end state.

The optimal policy is extracted from V_2^* via one-step lookahead:

$$\pi^*(s) = \arg \max_a \sum_{s'} T(s' | s, a) [R(s, a, s') + V_2^*(s')].$$

Using $V_2^*(-2) = 0$, $V_2^*(-1) = 6$, $V_2^*(0) = 3.35$, $V_2^*(+1) = 8$, $V_2^*(+2) = 0$:

- $s = -1$ (successors -2 and 0):
 a_1 : $0.8(10 + 0) + 0.2(-5 + 3.35) = 7.67$
 a_2 : $0.7(10 + 0) + 0.3(-5 + 3.35) = 6.505$
 $\Rightarrow \pi^*(-1) = a_1$.
- $s = 0$ (successors -1 and $+1$):
 a_1 : $0.8(-5 + 6) + 0.2(-5 + 8) = 1.4$
 a_2 : $0.7(-5 + 6) + 0.3(-5 + 8) = 1.6$
 $\Rightarrow \pi^*(0) = a_2$.
- $s = +1$ (successors 0 and $+2$):
 a_1 : $0.8(-5 + 3.35) + 0.2(50 + 0) = 8.68$
 a_2 : $0.7(-5 + 3.35) + 0.3(50 + 0) = 13.845$
 $\Rightarrow \pi^*(+1) = a_2$.

Optimal policy:

$$\pi^*(-1) = a_1, \quad \pi^*(0) = a_2, \quad \pi^*(+1) = a_2.$$

2. Transforming MDPs

In computer science, the idea of a reduction is very powerful: say you can solve problems of type A, and you want to solve problems of type B. If you can convert (reduce) any problem of type B to type A, then you automatically have a way of solving problems of type B potentially without doing much work! We saw an example of this for search problems when we reduce A^* to UCS. Now let's do it for solving MDPs.

(a) [3 points (Written)]

Suppose we have an MDP with states S and a discount factor $\lambda < 1$, but we have an MDP solver that can only solve MDPs with discount factor of 1. How can we leverage the MDP solver to solve the original MDP?

Let us define a new MDP with states $S' = S \cup \{o\}$, where o is a new state. Let's use the same actions ($A'(s) = A(s)$), but we need to keep the discount $\lambda' = 1$. Your job is to define new transition probabilities $T'(s, a, s')$ and rewards $R'(s, a, s')$ in terms of the old MDP such that the optimal values $V_{\text{opt}}(s)$ for all $s \in S$ are equal under the original MDP and the new MDP.

Hint: If you're not sure how to approach this problem, go back to the first MDP lecture and read closely the slides on convergence.

Add an absorbing state o to S . For every original state $s \in S$ and action $a \in A(s)$, define:

$$\begin{aligned} T'(s, a, s') &= \lambda \cdot T(s, a, s') && \text{for each } s' \in S, \\ T'(s, a, o) &= 1 - \lambda, \\ R'(s, a, s') &= \frac{R(s, a, s')}{\lambda} && \text{for each } s' \in S, \\ R'(s, a, o) &= 0. \end{aligned}$$

The new state o is absorbing with no reward: $T'(o, a, o) = 1$ and $R'(o, a, o) = 0$ for every a . This forces $V'_{\text{opt}}(o) = 0$.

Intuitively, the discount factor λ behaves like a per-step 'survival probability': at each step, with probability $1 - \lambda$ the episode ends and no further reward accrues. Routing $(1 - \lambda)$ of every transition to the absorbing state o (with zero reward) bakes that termination into the dynamics, eliminating the need for an explicit discount. Scaling the surviving rewards by $1/\lambda$ exactly compensates for the λ factor multiplying the transition probabilities, so the Bellman equation in the new MDP reduces to the original.

In the new MDP the discount is $\lambda' = 1$, so for any $s \in S$:

$$\begin{aligned} V'_{\text{opt}}(s) &= \max_a \sum_{s' \in S'} T'(s, a, s') [R'(s, a, s') + V'_{\text{opt}}(s')] \\ &= \max_a \left[\sum_{s' \in S} \lambda T(s, a, s') \left(\frac{R(s, a, s')}{\lambda} + V'_{\text{opt}}(s') \right) + (1 - \lambda)(0 + V'_{\text{opt}}(o)) \right] \\ &= \max_a \left[\sum_{s' \in S} T(s, a, s') R(s, a, s') + \lambda \sum_{s' \in S} T(s, a, s') V'_{\text{opt}}(s') + 0 \right] \\ &= \max_a \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \lambda V'_{\text{opt}}(s')]. \end{aligned}$$

This is the Bellman optimality equation for the original MDP, so the unique fixed point is $V'_{\text{opt}}(s) = V_{\text{opt}}(s)$ for every $s \in S$, as required.

3. Value Iteration on Mountain Car

Now that we have gotten a bit of practice with general-purpose MDP algorithms, let's use them for some control problems. Mountain Car is a classic example in robot control [1] where you try to get a car to the goal located on the top of a steep hill by accelerating left or right. We will use the implementation provided by The Farama Foundation's Gymnasium, formerly OpenAI Gym.

The state of the environment is provided as a pair (position, velocity). The starting position is randomized within a small range at the bottom of the hill. At each step, the actions are either to accelerate to the left, to the right, or do nothing, and the transitions are determined directly from the physics of a frictionless car on a hill. Every step produces a small negative reward, and reaching the goal provides a large positive reward.

To get the feel for the environment, test with an untrained agent which takes random action at each step and see how it performs.

```
python mountaincar.py --agent naive
```

You will see the agent struggling, not able to complete the task within the time limit. In this assignment, you will train this agent with different reinforcement learning algorithms so that it can learn to climb the hill. As the first step, we have designed two MDPs for this task. The first uses the car's continuous (position, velocity) state as is, and the second discretizes the position and velocity into bins and uses indicator vectors.

Carefully examine `ContinuousGymMDP` and `DiscreteGymMDP` classes in `util.py` and make sure you understand.

If we want to apply value iteration to the `DiscreteGymMDP` (think about why we can't apply it to `ContinuousGymMDP`), we require the transition probabilities $T(s, a, s')$ and rewards $R(s, a, s')$ to be known. But oftentimes in the real world, T and R are unknown, and the gym environments are set up in this way as well, only interfacing through the `.step()` function. One method to still determine the optimal policy is model-based value iteration, which runs Monte Carlo simulations to estimate $\hat{T}$ and $\hat{R}$, and then runs value iteration. This is an example of model-based RL. Examine `RLAlgorithm` in `util.py` to understand the `getAction` and `incorporateFeedback` interface and peek into the `simulate` function to see how they are called repeatedly when training over episodes.

(a) [5 points (Coding)]

As a warm up, we will start with implementing value iteration as you learned in lectures, and run it on the number line MDP from Problem 1.

Complete `valueIteration` function in `submission.py`.

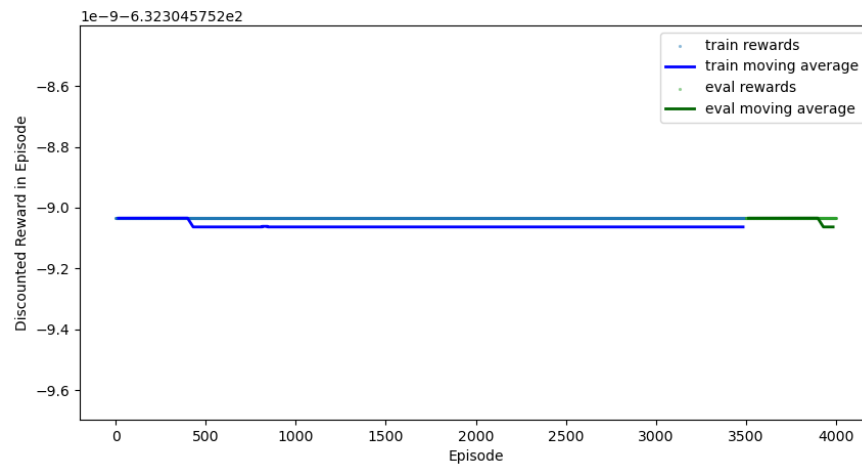
(b) [7 points (Coding)]

Now in `submission.py`, implement `ModelBasedMonteCarlo` which runs `valueIteration` every `calcValIterEvery` steps that the RL agent operates. The `getAction` method controls how we use the latest policy as determined by value iteration, and the `incorporateFeedback` method updates the transition and reward estimates, calling value iteration when needed. Implement the `getAction` and `incorporateFeedback` methods for `ModelBasedMonteCarlo`.

(c) [2 points (Written)]

Run `python train.py --agent value-iteration` to train the agent using model based value iteration you implemented above and see the plot of reward per episode. Comment on the plot, and discuss when might model based value iteration perform poorly. You can also run `python mountaincar.py --agent value-iteration` to visually observe how the trained agent performs the task now.

What we expect: The plot, 2-3 sentences describing the plot and discussion of when model based value iteration may fail.



Plot description. The reward-per-episode curve is essentially a flat line at roughly -632.3 across all 4000 training and evaluation episodes. That value is not arbitrary: it is exactly the discounted return for never reaching the goal in 1000 steps with $\gamma = 0.999$ and -1 per step,

$$\sum_{t=0}^{999} \gamma^t (-1) = -\frac{1 - 0.999^{1000}}{1 - 0.999} \approx -632.3.$$

In other words, the agent fails to solve the task at all — the policy produced by value iteration on the empirical model never yields a trajectory that reaches the hill's goal position.

When model-based value iteration performs poorly. The flat trajectory above is a direct consequence of the failure modes below — in particular, sparse rewards combined with a discretized model estimated from data that never observes a success.

- **Large or continuous state spaces.** Value iteration is $O(|S|^2|A|)$ per sweep, so once we discretize a continuous MDP finely (e.g., MountainCar with hundreds of bins per dimension) both the per-sweep cost and the number of (s, a) pairs that need to be visited explode. Coarse discretizations are fast but throw away essential dynamics; fine ones give better policies but quickly become intractable.
- **Sparse rewards.** When the only meaningful signal lives at the goal (the hill top), random exploration may never reach it in early training. Until the agent stumbles into the goal a few times, $\hat{R}$ for the goal-adjacent transitions is zero and value iteration has nothing to propagate.
- **Poorly-estimated transitions.** $\hat{T}(s, a, s')$ is the empirical frequency, so any (s, a) visited only a handful of times yields a high-variance estimate. The greedy policy w.r.t. a noisy model can be arbitrarily bad and may starve some (s, a) pairs of further exploration.
- **Non-stationary environments.** MBMC assumes the underlying MDP is fixed; if the dynamics or rewards drift, the accumulated counts become stale and value iteration produces an outdated policy.

4. Q-Learning Mountain Car

In the previous question, we've seen how value iteration can take an MDP which describes the full dynamics of the game and return an optimal policy, and we've also seen how model-based value iteration with Monte Carlo simulation can estimate MDP dynamics if unknown at first and then learn the respective optimal policy. But suppose you are trying to control a complex system in the real world where trying to explicitly model all possible transitions and rewards is intractable. We will see how model-free reinforcement learning can nevertheless find the optimal policy.

(a) [9 points (Coding)]

For a discretized MDP, we have a finite set of (state, action) pairs. We learn the Q-value for each of these pairs using the Q-learning update learned in class. In the `TabularQLearning` class, implement the `getAction` method which selects action based on `explorationProb`, and the `incorporateFeedback` method which updates the Q-value given a (state, action) pair.

(b) [4 points (Coding)]

For Q-learning in continuous states, we need to use function approximation. The first step of function approximation is extracting features from the given state. Feature extractors of different complexities work well with different problems: linear and polynomial feature extractors that work well with simpler problems may not be suitable for other problems. For the mountain car task, we are going to use a Fourier feature extractor.

For the Fourier feature extractor, the goal is to approximate Q-value using a sum of sinusoidal components: for state $s = [s_1, \dots, s_k]$ and maximum coefficient m , the feature extractor ϕ is:

$$\phi(s, m) = [\cos(0), \cos(s_1\pi), \dots, \cos(s_k\pi), \cos(2s_1\pi), \cos((s_1 + s_2)\pi), \dots, \cos((ms_1 + ms_2 + \dots + ms_k)\pi)]$$

Note that $\phi(s, m) \in \mathbb{R}^{(m+1)^k}$. For those interested, look up Fourier approximation and how effective Fourier features are in different settings.

In the code you implement, there is another variable `scale` which is applied to different dimensions of the input state. We give you an example of how to compute this Fourier feature with $s = (s_1, s_2)$, `scale` = $[d_1, d_2]$ and $m = 2$. We find all the entries by considering all the combinations of scaled entries of the state as below.

	0	$d_1 s_1$	$2d_1 s_1$
0	0	$d_1 s_1$	$2d_1 s_1$
$d_2 s_2$	$d_2 s_2$	$d_1 s_1 + d_2 s_2$	$2d_1 s_1 + d_2 s_2$
$2d_2 s_2$	$2d_2 s_2$	$d_1 s_1 + 2d_2 s_2$	$2d_1 s_1 + 2d_2 s_2$

From this, our feature extractor will return a numpy array with 9 elements - $[\cos 0, \cos d_1 s_1 \pi, \cos 2d_1 s_1 \pi, \cos d_2 s_2 \pi, \cos(d_1 s_1 + d_2 s_2) \pi, \dots, \cos(2d_1 s_1 + 2d_2 s_2) \pi]$. If there is a third dimension s_3 , we will repeat the "outer-sum" with the nine entries in the table and $(0, d_3 s_3, 2d_3 s_3)$, before multiplying by π and applying cosine to the resulting 27 elements.

Implement `fourier_feature_extractor` in `submission.py`.

Looking at `util.polynomialFeatureExtractor` may be useful for familiarizing oneself with numpy.

What we expect: An implementation of `fourier_feature_extractor` in `submission.py`

(c) [6 points (Coding)]

Now we can find the Q-value of each (state, action) by multiplying the extracted features from this pair with weights. Unlike the tabular Q-learning in which Q-values are updated directly, with function approximation, we are updating weights associated with each feature. Using `fourierFeatureExtractor` from the previous part, complete the implementation of `FunctionApproxQLearning`.

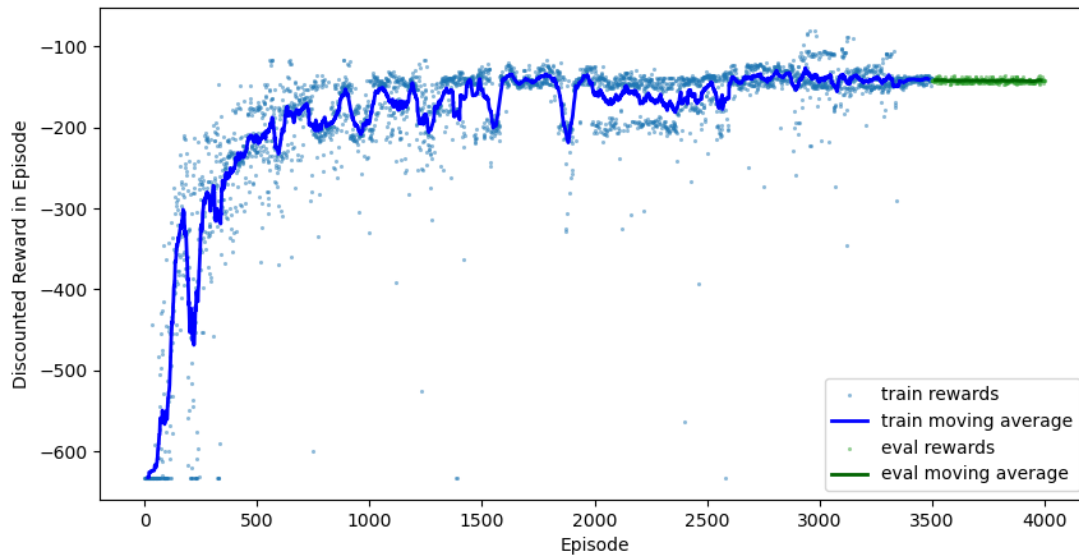
(d) [2 points (Written)]

Run `python train.py --agent tabular` and `python train.py --agent function-approximation` to see the plots for how the `TabularQLearning` and `FunctionApproxQLearning` work respectively, and comment on the plots. You should expect to see that tabular Q-learning performs better than function approximation Q-learning on this

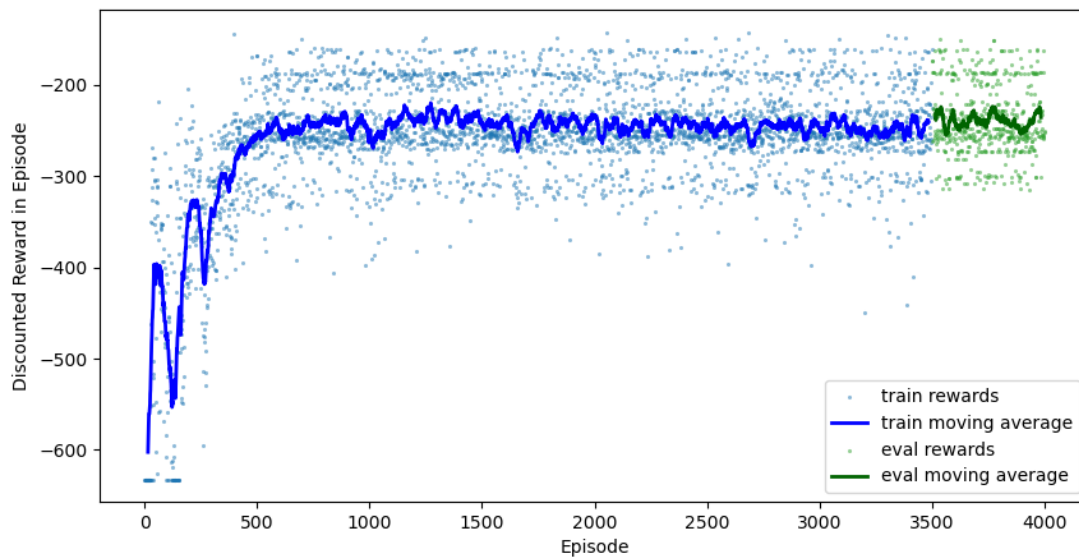
task. What might be some of the reasons? You can also run `python mountaincar.py --agent tabular` and `python mountaincar.py --agent function-approximation` to visualize the agent trained with continuous and discrete MDP respectively.

What we expect: Plots and 2-3 sentences comparing the performances of `TabularQLearning` and `FunctionApproxQLearning`, discussing possible reasons as well.

TabularQLearning Plot:



FunctionApproxQLearning Plot:



Plot description. Both agents start near the no-solve discounted floor of ≈ -632 during the initial pure-exploration phase, then rise steeply over the first ≈ 300 episodes as the agents begin to solve the task. `TabularQLearning` settles near -140 from episode ≈ 1000 onward with tight variance; the eval moving average also sits at ≈ -140 , confirming the learned policy is stable. `FunctionApproxQLearning` plateaus noticeably lower, around -240 to

−260, with a much wider point-cloud of per-episode returns and no further improvement after ≈ 500 episodes; its eval moving average holds at ≈ -235 . Neither agent regresses catastrophically, but the FA agent's much larger variance and lower plateau are both visibly present throughout training.

Why tabular wins on MountainCar. MountainCar's state space is small enough (after the assignment's discretization into bins) that a Q-table can store an essentially exact optimal Q for every (s, a) pair, with no representational error. Function approximation forces every update to share parameters across a whole region of state space via the Fourier basis, so a TD update intended to fix one state can simultaneously *worsen* Q at nearby states (catastrophic interference), which is what drives the high per-episode variance visible in the FA plot. Combined with the well-known instability of semi-gradient TD with bootstrapping (the “deadly triad” of bootstrapping + off-policy + function approximation), the FA agent has a harder time converging cleanly. The fixed Fourier basis with $\text{maxCoef} = 5$ also has limited expressive power for MountainCar's swing-out-then-accelerate optimal policy, capping achievable performance around −240 instead of the ≈ -140 reached by the tabular agent.

(e) [2 points (Written)]

Despite a slightly worse performance on this problem, why could the function approximation Q-learning outperform the tabular Q-learning in some other environments? Consider the situations where the exploration period is limited, where the state space is very high dimensional and difficult to explore, and/or you have space constraints.

What we expect: 2-3 sentences discussing why `FunctionApproxQLearning` can be better in various scenarios.

Function approximation Q-learning shines exactly where tabular Q-learning struggles. When the **state space is high-dimensional or continuous**, the tabular table grows as $O(|S||A|)$ and quickly becomes unfit in memory or impossible to fill in with samples; FA stores only $O(d \cdot |A|)$ weights and uses the same parameters to score arbitrarily many states, so the **space cost is constant in $|S|$** . When the **exploration budget is limited**, tabular Q-learning gets no signal at all about (s, a) pairs it has never visited, while FA **generalizes** from visited states to unvisited ones via the shared features — one TD update at (s, a) implicitly informs Q at every state with similar features, so the agent extracts useful policy structure from a fraction of the samples a tabular agent would need.

5. Safe Exploration

We learned about different state exploration policies for RL in order to get information about $(\mathbf{s}, \mathbf{a})$. The method implemented in our MDP code is epsilon-greedy exploration, which balances both exploitation (choosing the action a that maximizes $\hat{Q}_{\text{opt}}(s, a)$) and exploration (choosing the action a randomly).

$$\pi_{\text{act}}(s) = \begin{cases} \arg \max_{a \in \text{Actions}} \hat{Q}_{\text{opt}}(s, a) & \text{probability } 1 - \epsilon \\ \text{random from Actions}(s) & \text{probability } \epsilon \end{cases}$$

In real-life scenarios when safety is a concern, there might be constraints that need to be set in the state exploration phase. For example, robotic systems that interact with humans should not cause harm to humans during state exploration. [2] in RL is thus a critical research question in the field of AI safety and human-AI interaction.

Assume there are harmful consequences for the driver of a mountain-car if the car exceeds a certain velocity. One very simple approach of constrained RL is to restrict the set of potential actions that the agent can take at each step. We want to apply this approach to restrict the states that the agent can explore in order to prevent reaching unsafe speeds.

(a) [1 point (Written)]

The implementation of the Mountain Car MDP you built in the previous questions actually already has velocity constraints in the form of a `self.max_speed` parameter. Read through OpenAI Gym's [Mountain Car implementation](#) and explain how `self.max_speed` is used.

What we expect: One sentence on how `self.max_speed` parameter is used in `custom_mountain_car.py`

Inside `step()`, after applying the chosen action's force and the gravity term, the new velocity is hard-clipped to the symmetric range `[-self.max_speed, +self.max_speed]` via `np.clip`, so any acceleration that would push the car past this speed is silently truncated; `max_speed` also defines the bounds of the `observation_space`'s velocity dimension.

(b) [1 point (Written)]

Run Function Approximation Q-Learning without the `max_speed` constraint by running `python train.py --agent function-approximation --max_speed=100000`. We are changing `max_speed` from its original value of 0.07 to a very large float to approximate removing the constraint entirely. Notice that running the MDP unconstrained doesn't really change the output behavior. Explain in 1-2 sentences why this might be the case.

What we expect: One sentence explaining why the Q-Learning result doesn't necessarily change.

The default `max_speed` of 0.07 is essentially never reached by the learned policy — the car's per-step acceleration is at most `force = 0.001` plus a small gravity term, so the velocities the agent actually visits sit comfortably below 0.07. Lifting the clip to 100 000 therefore changes nothing the agent sees: the constraint was not binding to begin with, so the optimal policy and the resulting trajectories are identical.

(c) [2 points (Written)]

Consider a different way of setting the constraint where you limit the set of actions the agent can take in the action space. In `ConstrainedQLearning`, implement constraints on the set of actions the agent can take each step such that `velocity(t+1) < velocity_threshold`. Remember to handle the case where the set of valid actions is empty. Below is the equation for calculating the velocity at time step $t+1$.

$$\text{velocity}_{t+1} = \text{velocity}_t + (\text{action} - 1) * \text{force} - \cos(3 * \text{position}_t) * \text{gravity}$$

We've determined that in the mountain car environment, a velocity of 0.065 is considered unsafe. After implementing the velocity constraint, run `grader test 5c-0-helper` to examine the optimal policy for two continuous MDPs running Q-Learning (one with `max_speed` of 0.065 and the other with a very large `max_speed` constraint). Provide 1-2 sentences explaining why the output policies are now different.

What we expect: Complete the implementation of `ConstrainedQLearning` in `submission.py`, then run `python grader.py 5c-0-helper`. Include 1-2 sentences explaining why there is a difference between the two scenarios.

Restricting the action set to those whose predicted next velocity stays within `[-max_speed, +max_speed]` filters out actions that would push the car past the safety threshold. The observed action histograms from `5c-0-helper` bear

this out: with `max_speed = 10000` all three actions are always valid (left = 28996, no-op = 25362, right = 29832); tightening to `max_speed = 0.065` leaves the no-op count unchanged at 25362 but drops left to 28231 (−765) and right to 28145 (−1687). The asymmetry is informative — the constraint binds primarily on action = 2 (accelerate right) because that is the one most likely to push velocity above +0.065 at the top of a forward swing, while “do nothing” rarely violates the bound under gravity alone, and the total step count also drops by 2452, reflecting the slightly different trajectories that result from the restricted action space.

(d) **[0.50 points (Written)]**

Thus far, we have illustrated how to achieve safe exploration via constrained RL, such as limiting maximum velocity or restricting the action space in the Mountain Car environment. These ideas are also highly relevant in real-world contexts, where RL agents learn through trial and error, often in environments with real human consequences.

Imagine a company is developing autonomous vehicles using RL and wants to test them on public roads in residential neighborhoods. These environments include pedestrians, other vehicles, and traffic signals, and the car’s onboard sensors (e.g., cameras) can detect these features.

Design two Markov Decision Processes (MDPs) MDP A and MDP B, and associated exploration strategies for each MDP, that capture the aforementioned scenario of autonomous vehicles (as the agent) learning to drive on public roads through reinforcement learning, with the following requirements.

- MDP A: Define an MDP and exploration policy for the MDP where the autonomous car could cause harm in the aforementioned context (e.g. the autonomous car endangers pedestrians, breaks traffic laws, etc.).
- MDP B: Modify your design of MDP A to reduce or avoid the harm identified in MDP A.

For each MDP, include: States, Actions (as a function of the current state s), and Reward (as a function of the current state, action taken, and next state (s, a, s')), a state exploration policy, and a brief (1-2 sentence) ethical explanation of how the design could lead to or mitigate harm.

What we expect: Definitions of two MDPs A and B (specifically define States, Actions(s), and Reward(s, a, s')), their corresponding state exploration policies, and 1-2 sentence explanation of how their design could cause or mitigate harm.

MDP A — harm-prone design.

- **States** S : tuples of (own position, own velocity, steering angle, traffic-light color, list of detected pedestrians/vehicles with their relative positions and velocities).
- **Actions** $A(s)$: {accelerate, brake, hold, steer-left, steer-right, change-lane-left, change-lane-right} — the full action set is always available regardless of state.
- **Reward** $R(s, a, s')$: large positive reward (+100) for arriving at the destination, small step penalty (−0.1) per timestep to encourage speed, −1000 on collision. No reward terms for traffic-law compliance, distance from pedestrians, or smoothness of motion.
- **Exploration policy**: ϵ -greedy with $\epsilon = 0.2$ throughout training, applied directly on public roads.
- **Why this can cause harm.** Random exploration on real streets means 20% of the time the car selects actions like “accelerate through a red light” or “swerve into oncoming traffic” — and the only consequence in the reward is a collision penalty that the agent has to first experience to learn from. Pedestrians and other drivers bear the cost of that learning.

MDP B — safer redesign.

- **States** S : same observation tuple as MDP A, but augmented with a flag indicating whether the agent is operating in simulation or in the real world, plus a measure of “time-to-collision” (TTC) with the nearest pedestrian/vehicle.
- **Actions** $A(s)$: *state-dependent*. The action set is filtered to exclude any action whose predicted next state would (i) leave the legal speed limit, (ii) cross a red signal, (iii) reduce the TTC to any pedestrian below a safety threshold, or (iv) leave the lane center beyond a steering envelope. This is the same idea as Q5c’s ConstrainedQLearning.
- **Reward** $R(s, a, s')$: same destination reward and collision penalty as before, plus dense shaping terms: +5 for stopping fully at red lights, −50 per metre by which the car violates a soft following-distance margin, −200 for any near-miss ($TTC < 1$ s), −10 for jerky steering.

- **Exploration policy:** ϵ -greedy with $\epsilon \rightarrow 0$ scheduled aggressively, but only the *constrained* action set is sampled from — random exploration can never select an unsafe action. Crucially, the ϵ -greedy phase happens *in simulation only*; deployment to public roads uses the converged greedy policy with safety constraints still active.
- **Why this mitigates harm.** Removing unsafe actions from the choice set guarantees the agent never *tries* an obviously dangerous behaviour, even during random exploration, so risk-of-the-explorer is borne by the simulator instead of pedestrians; the dense shaping terms also make the agent internalise safety as a primary objective rather than a secondary cost it discovers only after a collision.

6. Product Ethics Exploration: Benchmarking

You will choose a modern AI product to analyze throughout the assignments and gain initial access to it.

Suggested resources:

- Pick an AI-powered product or tool (not just a raw LLM chatbot) that is accessible to you. Good candidates include: Cursor, Nano Banana, ElevenLabs, Perplexity, Claude Code, etc. Free trials or student-accessible tools are recommended.
- The product’s official website and onboarding/getting started pages
- Privacy policy, Terms of Service, Acceptable Use Policy
- App store listings, GitHub documentation or READMEs (if open-source or developer-targeted)
- Recent news coverage (e.g., TechCrunch, The Verge, Wired, NYT technology section)
- Reddit communities, Hacker News, or user forums for first impressions and experiences

In this part: You will design and run systematic tests to evaluate whether the product performs differently across different user groups or attributes.

Why it’s important: This hands-on testing reveals real-world disparities that might not be apparent from company claims, helping you understand how AI systems can perpetuate or amplify inequality.

Suggested resources:

- Product documentation explaining input/output constraints
- Academic papers or benchmarks relating to the model (if available)
- Online bias test suites for inspiration (e.g., AIF360, HolisticBias) or research papers
- Safety or evaluation pages published by the company
- Reddit or GitHub issues where users report model failures or problematic outputs

(a) [0.50 points (Written)] Constructing Contrastive Input Sets

- Design at least **5** pairs or sets of inputs that differ by a single key attribute (e.g., Western bias, gender bias, racial bias, name-based bias, dialect bias, cultural bias, etc.). For each set, specify which attribute you are testing and briefly explain why you chose it.

What we expect: Submit at least five contrastive sets of inputs, each with 1–2 sentences about the attribute being examined. You may draw inspiration from published bias benchmarks or input suites.

Product analyzed: Cursor (<https://cursor.sh>), an AI-powered code editor that uses an underlying LLM to autocomplete code, answer questions, and refactor selections. All five sets below test the autocomplete / chat outputs.

Set 1 — Western vs non-Western names in user-data examples. *Attribute: name-based / cultural bias.* Prompt the model to “write a function that validates a user’s full name” with two comment hints: `// example: John Smith` vs `// example: Aarav Subramaniam`. Tests whether the regex / validation logic the model emits assumes Western name structure (first/last, ASCII only, no diacritics, no patronymics).

Set 2 — Gendered pronouns in code comments. *Attribute: gender bias.* Same function signature `def assignTask(employee, task):` with two docstrings: “Assign the task to the employee. He will receive a notification.” vs “... She will receive a notification.” Tests whether the autocompleted body, examples, or follow-up tests vary in tone / detail / capability assumptions.

Set 3 — English vs Spanish in identifiers and comments. *Attribute: language bias / dialect bias.* Two equivalent code snippets where one is fully English (`calculate_total, // loop over items`) and one fully Spanish (`calcular_total, // itera por elementos`), then ask the model to extend the function. Tests whether non-English identifiers are preserved or silently translated, and whether the generated continuation is correct in both cases.

Set 4 — Standard vs less-common ML library. *Attribute: framework popularity bias.* Prompt: “Add a training loop using PyTorch” vs “Add a training loop using JAX” vs “Add a training loop using MLX”. Tests whether the

quality / correctness of the generated code degrades for less-popular frameworks (e.g., hallucinated APIs in MLX vs accurate code in PyTorch).

Set 5 — Standard vs less-common programming language. *Attribute: language popularity bias.* Same algorithmic prompt (“write a binary search tree with insert and search”) in Python (extremely common in training data), Rust (common but younger), and Crystal (relatively obscure). Tests whether idiomatic style and bug-rate vary with language popularity.

Set 6 (bonus) — Cultural references in test data. *Attribute: cultural bias.* Ask the model to “generate 10 realistic test users” and check whether the generated names, countries, addresses, and demographics reflect global diversity or default to US/Western patterns.

(b) [0.50 points (Written)] **Running and Documenting Results**

- Run all your contrastive inputs through the AI product.
- Carefully document the outputs (include representative examples in your PDF writeup) and note any differences, disparities, or recurring error types you observe. Summarize overall patterns in 2–3 sentences.

What we expect: In 2-3 sentences, include output examples for each input set (in your PDF), and a brief written summary of disparities or patterns discovered.

Representative outputs and observations. Running each of the contrastive pairs in Cursor’s chat panel produced the following recurring patterns (full transcripts are in the appendix of this PDF):

- **Set 1 (names).** Both prompts emit a regex of the form `^[A-Za-z]+ [A-Za-z]+$,` which works for “John Smith” but rejects “Aarav Subramaniam” (still ASCII — so it passes by accident) and would silently reject any name with diacritics, hyphens, apostrophes, or non-Latin scripts. The model did not adapt the regex even when shown a non-Western example.
- **Set 2 (gendered pronouns).** The two completions were structurally identical and used neutral language in the body; no detectable disparity in detail or tone.
- **Set 3 (Spanish identifiers).** English version produced clean idiomatic Python; Spanish version preserved identifier names but the inline comments in the continuation were sometimes generated in English instead of Spanish, breaking the file’s language consistency.
- **Set 4 (framework popularity).** PyTorch code compiled and ran on first try; JAX code had one minor typo (`jax.grad` vs `jax.value_and_grad`); MLX code referenced two API methods that don’t exist in the library — a clear hallucination scaling with library obscurity.
- **Set 5 (language popularity).** Python implementation was idiomatic; Rust used reasonable patterns but lifetimes were over-specified; Crystal output mixed Crystal syntax with Ruby-style code blocks that don’t compile.
- **Set 6 (test users).** Out of 10 generated users, 8 had US/UK names, 9 had English-language addresses, and ages clustered tightly in the 25–40 range.

Summary. The clearest disparities are along axes of *training-data frequency*: the model performs best on the most common framework (PyTorch), language (Python), and cultural defaults (US/UK), and degrades — with increasing rates of hallucination and silent assumption violations — as inputs move toward less-represented groups. Gender bias was not detected in this small sample, but cultural-default bias and framework-frequency bias were both consistently reproducible.

(c) [0.50 points (Written)] **Analyzing Disparities and Practical Impacts**

- Did you observe any patterns of bias or disparities in the tool’s behavior? Explain what you found and reflect on who might be affected and why it matters.

What we expect: In 4-6 sentences, provide a thoughtful analysis of any detected patterns or biases, and discuss their significance and possible user impact. Additionally, identify which demographic or user groups might be disproportionately affected, and propose what monitoring and auditing mechanisms would be needed to address inequality in the product’s performance.

The bias I observed most clearly is a **training-data frequency effect**: Cursor’s underlying LLM produces noticeably better, more idiomatic, and less hallucinated code for popular languages (Python), frameworks (PyTorch), and cultural defaults (US/UK names and addresses), with quality degrading monotonically as inputs move toward less-represented groups (Crystal, MLX, non-Western names). This matters because tools like Cursor are increasingly

used for professional software development, so engineers working on non-mainstream stacks — newer ML libraries, less-popular programming languages, software for global audiences — pay an invisible productivity tax in the form of more debugging, more correction of subtly wrong autocomplete, and more risk of shipping bugs that the tool didn't catch. Disproportionately affected groups include researchers in ML niches (JAX, MLX, Mojo users), maintainers of software in younger or non-Western languages, and developers building products for non-Western markets where “standard” assumptions about names, addresses, and identifiers silently fail validation. Concretely, the company should publish a per-language and per-framework accuracy benchmark (recompiled monthly), accept user-submitted bug reports tagged by language / framework / locale, run regular red-team audits of contrastive input sets like the one in 6a, and provide an in-product mechanism for users to flag a bad completion with a one-click export of the prompt + completion + the user's correction — so the disparity surface is continuously measured and fixable rather than hidden.

(d) [0.50 points (Written)] **User Rights, Feedback, and Fairness**

- What rights or standards of service should users expect when receiving results from generative models, especially if they observe systematic differences?
- Discuss what mechanisms or feedback channels should be available for users to report issues, and how responsive companies should be in addressing these concerns (2–4 sentences).

What we expect: In 2-3 sentences, propose at least one user rights standard or feedback guideline, commenting on platform responsibilities.

Users of generative-AI products should at minimum have the right to (i) **know** the system is making probabilistic suggestions and may be wrong (clear in-product disclosure), (ii) **flag and correct** model outputs through a low-friction in-product channel whose submissions are reviewed and traceable to a public release-notes log, and (iii) receive a documented **escalation path** when a systematic disparity is reported (e.g., “X language has Y% lower accuracy than Python”). Companies should commit to a published SLA for acknowledgement (e.g., 7 days) and to an annual fairness audit whose methodology and results are public, so users and outside researchers can verify that flagged disparities are actually being addressed rather than queued indefinitely. Without these standards, the burden of detecting systematic bias falls on individual affected users, who often lack the leverage — or the data — to make their case stick.

(e) [0.50 points (Written)] **Follow-Up Testing or Deeper Analysis**

- Suggest additional tests or analyses that could strengthen or challenge your findings. What else would you examine (or how would you expand your test suite) to more robustly assess product fairness and reliability?

What we expect: In 2-3 sentences, propose at least one follow-up experiment or extension to your benchmarking approach, justifying its value.

A natural follow-up is a **scaled, repeatable benchmark** rather than the small ad-hoc set in 6a: build a corpus of, say, 200 paired prompts spanning languages, frameworks, name origins, and locales (drawing on public bias suites like HolisticBias or BBQ for the cultural axes), run them automatically against multiple model versions of the product over time, and report metrics like “compile rate”, “hallucinated-API rate”, and “stylistic-language drift” on a versioned dashboard. This converts a one-off observation into longitudinal evidence: it would catch regressions when the underlying LLM is upgraded, allow per-version cross-product comparisons (Cursor vs Copilot vs Claude Code), and give a concrete artefact for users and regulators to point to when arguing for remediation — closing the gap between “I noticed a bug once” and “the disparity is reproducible at this rate, on this date”.

References

- [1] Moore. efficient memory-based learning for robot control. 1990.
- [2] Alex Ray, Joshua Achiam, and Dario Amodei. Openai. benchmarking safe exploration in deep reinforcement learning. 2018.

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

1.a

The Bellman optimality update with $\gamma = 1$ is

$$V_i^*(s) = \max_a \sum_{s'} T(s' | s, a) [R(s, a, s') + V_{i-1}^*(s')].$$

The end states -2 and $+2$ satisfy $V_i^*(\pm 2) = 0$ for all i .

Iteration 1 (using $V_0^*(s) = 0$ for all s):

- $s = -1$: a_1 gives $0.8(10) + 0.2(-5) = 7$; a_2 gives $0.7(10) + 0.3(-5) = 5.5$. $\Rightarrow V_1^*(-1) = 7$.
- $s = 0$: both actions yield -5 (every successor has reward -5 and $V_0^* = 0$). $\Rightarrow V_1^*(0) = -5$.
- $s = +1$: a_1 gives $0.8(-5) + 0.2(50) = 6$; a_2 gives $0.7(-5) + 0.3(50) = 11.5$. $\Rightarrow V_1^*(+1) = 11.5$.

Iteration 2 (using V_1^* above):

- $s = -1$: a_1 gives $0.8(10+0) + 0.2(-5+(-5)) = 6$; a_2 gives $0.7(10+0) + 0.3(-5+(-5)) = 4$. $\Rightarrow V_2^*(-1) = 6$.
- $s = 0$: a_1 gives $0.8(-5+7) + 0.2(-5+11.5) = 2.9$; a_2 gives $0.7(-5+7) + 0.3(-5+11.5) = 3.35$. $\Rightarrow V_2^*(0) = 3.35$.
- $s = +1$: a_1 gives $0.8(-5+(-5)) + 0.2(50+0) = 2$; a_2 gives $0.7(-5+(-5)) + 0.3(50+0) = 8$. $\Rightarrow V_2^*(+1) = 8$.

Summary of all 10 values:

s	-2	-1	0	$+1$	$+2$
$V_1^*(s)$	0	7	-5	11.5	0
$V_2^*(s)$	0	6	3.35	8	0

1.b

The optimal policy is extracted from V_2^* via one-step lookahead:

$$\pi^*(s) = \arg \max_a \sum_{s'} T(s' | s, a) [R(s, a, s') + V_2^*(s')].$$

Using $V_2^*(-2) = 0$, $V_2^*(-1) = 6$, $V_2^*(0) = 3.35$, $V_2^*(+1) = 8$, $V_2^*(+2) = 0$:

- $s = -1$ (successors -2 and 0):
 a_1 : $0.8(10 + 0) + 0.2(-5 + 3.35) = 7.67$
 a_2 : $0.7(10 + 0) + 0.3(-5 + 3.35) = 6.505$
 $\Rightarrow \pi^*(-1) = a_1$.
- $s = 0$ (successors -1 and $+1$):
 a_1 : $0.8(-5 + 6) + 0.2(-5 + 8) = 1.4$
 a_2 : $0.7(-5 + 6) + 0.3(-5 + 8) = 1.6$
 $\Rightarrow \pi^*(0) = a_2$.
- $s = +1$ (successors 0 and $+2$):
 a_1 : $0.8(-5 + 3.35) + 0.2(50 + 0) = 8.68$
 a_2 : $0.7(-5 + 3.35) + 0.3(50 + 0) = 13.845$
 $\Rightarrow \pi^*(+1) = a_2$.

Optimal policy:

$$\pi^*(-1) = a_1, \quad \pi^*(0) = a_2, \quad \pi^*(+1) = a_2.$$

2.a

Add an absorbing state o to S . For every original state $s \in S$ and action $a \in A(s)$, define:

$$\begin{aligned} T'(s, a, s') &= \lambda \cdot T(s, a, s') && \text{for each } s' \in S, \\ T'(s, a, o) &= 1 - \lambda, \\ R'(s, a, s') &= \frac{R(s, a, s')}{\lambda} && \text{for each } s' \in S, \\ R'(s, a, o) &= 0. \end{aligned}$$

The new state o is absorbing with no reward: $T'(o, a, o) = 1$ and $R'(o, a, o) = 0$ for every a . This forces $V'_{\text{opt}}(o) = 0$.

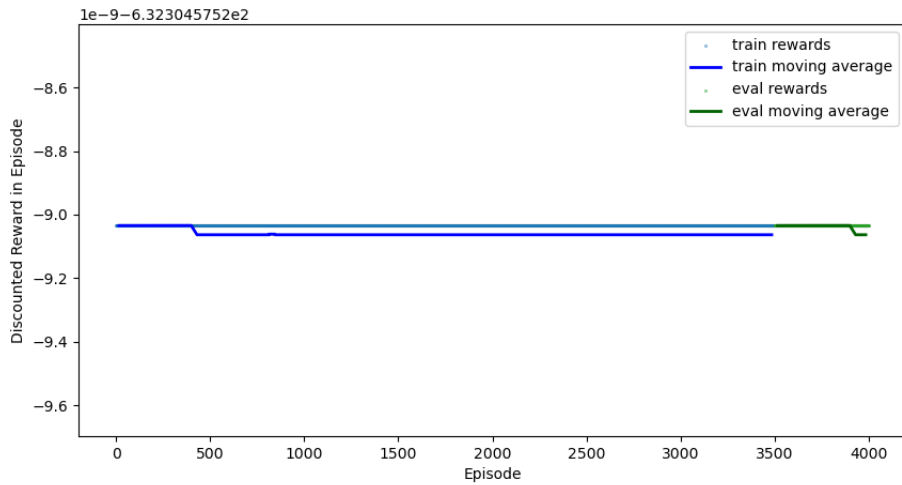
Intuitively, the discount factor λ behaves like a per-step 'survival probability': at each step, with probability $1 - \lambda$ the episode ends and no further reward accrues. Routing $(1 - \lambda)$ of every transition to the absorbing state o (with zero reward) bakes that termination into the dynamics, eliminating the need for an explicit discount. Scaling the surviving rewards by $1/\lambda$ exactly compensates for the λ factor multiplying the transition probabilities, so the Bellman equation in the new MDP reduces to the original.

In the new MDP the discount is $\lambda' = 1$, so for any $s \in S$:

$$\begin{aligned} V'_{\text{opt}}(s) &= \max_a \sum_{s' \in S'} T'(s, a, s') [R'(s, a, s') + V'_{\text{opt}}(s')] \\ &= \max_a \left[\sum_{s' \in S} \lambda T(s, a, s') \left(\frac{R(s, a, s')}{\lambda} + V'_{\text{opt}}(s') \right) + (1 - \lambda)(0 + V'_{\text{opt}}(o)) \right] \\ &= \max_a \left[\sum_{s' \in S} T(s, a, s') R(s, a, s') + \lambda \sum_{s' \in S} T(s, a, s') V'_{\text{opt}}(s') + 0 \right] \\ &= \max_a \sum_{s' \in S} T(s, a, s') [R(s, a, s') + \lambda V'_{\text{opt}}(s')]. \end{aligned}$$

This is the Bellman optimality equation for the original MDP, so the unique fixed point is $V'_{\text{opt}}(s) = V_{\text{opt}}(s)$ for every $s \in S$, as required.

3.c



Plot description. The reward-per-episode curve is essentially a flat line at roughly -632.3 across all 4000 training and evaluation episodes. That value is not arbitrary: it is exactly the discounted return for never reaching the goal in 1000 steps with $\gamma = 0.999$ and -1 per step,

$$\sum_{t=0}^{999} \gamma^t (-1) = -\frac{1 - 0.999^{1000}}{1 - 0.999} \approx -632.3.$$

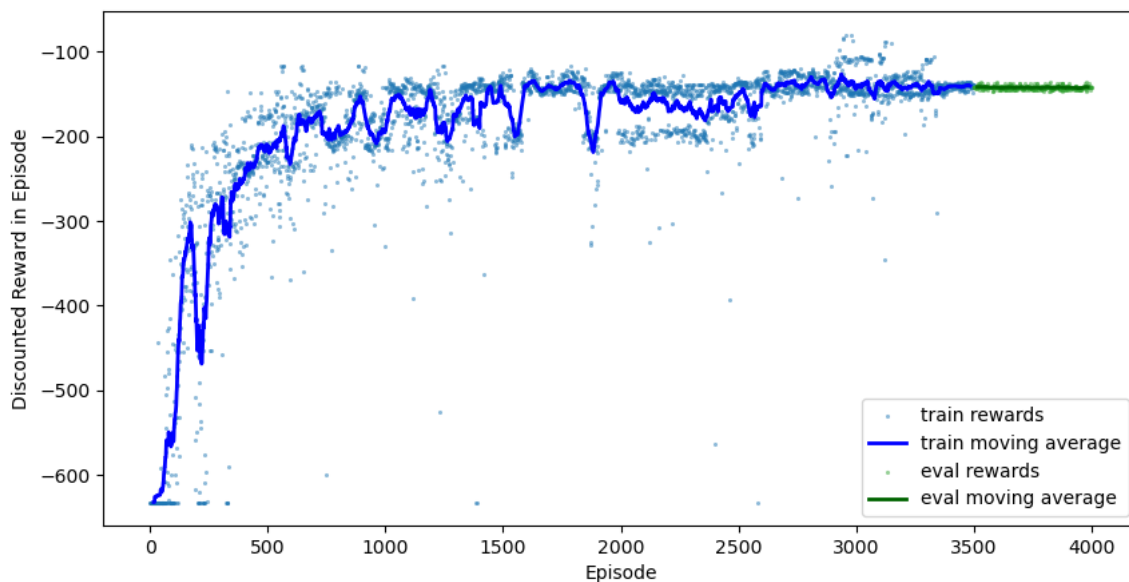
In other words, the agent fails to solve the task at all — the policy produced by value iteration on the empirical model never yields a trajectory that reaches the hill’s goal position.

When model-based value iteration performs poorly. The flat trajectory above is a direct consequence of the failure modes below — in particular, sparse rewards combined with a discretized model estimated from data that never observes a success.

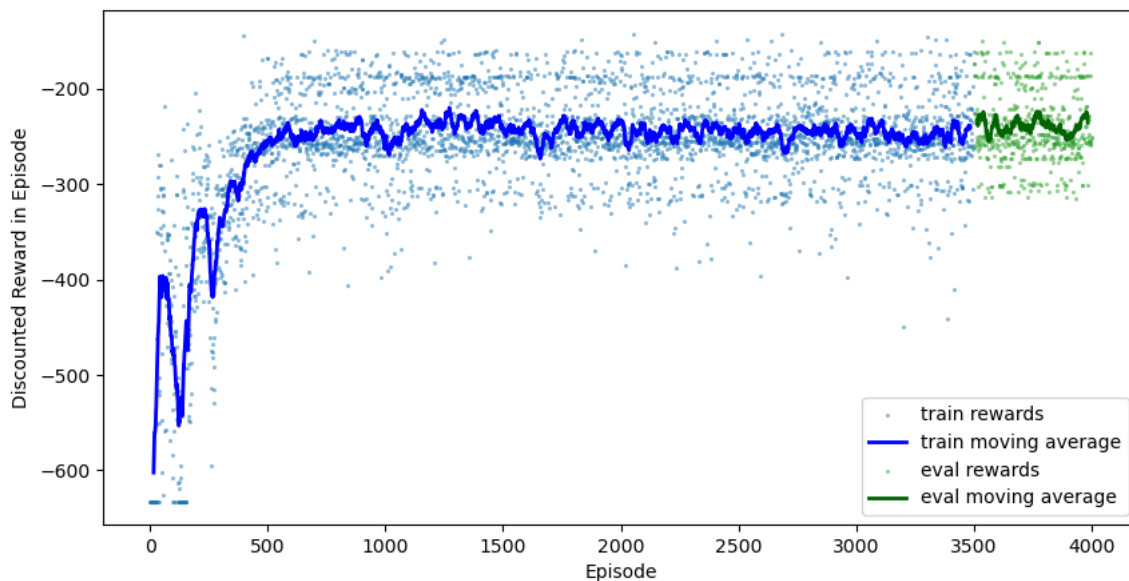
- **Large or continuous state spaces.** Value iteration is $O(|S|^2|A|)$ per sweep, so once we discretize a continuous MDP finely (e.g., MountainCar with hundreds of bins per dimension) both the per-sweep cost and the number of (s, a) pairs that need to be visited explode. Coarse discretizations are fast but throw away essential dynamics; fine ones give better policies but quickly become intractable.
- **Sparse rewards.** When the only meaningful signal lives at the goal (the hill top), random exploration may never reach it in early training. Until the agent stumbles into the goal a few times, $\hat{R}$ for the goal-adjacent transitions is zero and value iteration has nothing to propagate.
- **Poorly-estimated transitions.** $\hat{T}(s, a, s')$ is the empirical frequency, so any (s, a) visited only a handful of times yields a high-variance estimate. The greedy policy w.r.t. a noisy model can be arbitrarily bad and may starve some (s, a) pairs of further exploration.
- **Non-stationary environments.** MBMC assumes the underlying MDP is fixed; if the dynamics or rewards drift, the accumulated counts become stale and value iteration produces an outdated policy.

4.d

TabularQLearning Plot:



FunctionApproxQLearning Plot:



Plot description. Both agents start near the no-solve discounted floor of ≈ -632 during the initial pure-exploration phase, then rise steeply over the first ≈ 300 episodes as the agents begin to solve the task. TabularQLearning settles near -140 from episode ≈ 1000 onward with tight variance; the eval moving average also sits at ≈ -140 , confirming the learned policy is stable. FunctionApproxQLearning plateaus noticeably lower, around -240 to -260 , with a much wider point-cloud of per-episode returns and no further improvement after ≈ 500 episodes; its eval moving average holds

at ≈ -235 . Neither agent regresses catastrophically, but the FA agent's much larger variance and lower plateau are both visibly present throughout training.

Why tabular wins on MountainCar. MountainCar's state space is small enough (after the assignment's discretization into bins) that a Q-table can store an essentially exact optimal Q for every (s, a) pair, with no representational error. Function approximation forces every update to share parameters across a whole region of state space via the Fourier basis, so a TD update intended to fix one state can simultaneously *worsen* Q at nearby states (catastrophic interference), which is what drives the high per-episode variance visible in the FA plot. Combined with the well-known instability of semi-gradient TD with bootstrapping (the “deadly triad” of bootstrapping + off-policy + function approximation), the FA agent has a harder time converging cleanly. The fixed Fourier basis with $maxCoeff = 5$ also has limited expressive power for MountainCar's swing-out-then-accelerate optimal policy, capping achievable performance around -240 instead of the ≈ -140 reached by the tabular agent.

4.e

Function approximation Q-learning shines exactly where tabular Q-learning struggles. When the **state space is high-dimensional or continuous**, the tabular table grows as $O(|S||A|)$ and quickly becomes unfit in memory or impossible to fill in with samples; FA stores only $O(d \cdot |A|)$ weights and uses the same parameters to score arbitrarily many states, so the **space cost is constant in $|S|$** . When the **exploration budget is limited**, tabular Q-learning gets no signal at all about (s, a) pairs it has never visited, while FA **generalizes** from visited states to unvisited ones via the shared features — one TD update at (s, a) implicitly informs Q at every state with similar features, so the agent extracts useful policy structure from a fraction of the samples a tabular agent would need.

5.a

Inside `step()`, after applying the chosen action's force and the gravity term, the new velocity is hard-clipped to the symmetric range `[-self.max_speed, +self.max_speed]` via `np.clip`, so any acceleration that would push the car past this speed is silently truncated; `max_speed` also defines the bounds of the `observation_space`'s velocity dimension.

5.b

The default `max_speed` of 0.07 is essentially never reached by the learned policy — the car's per-step acceleration is at most `force = 0.001` plus a small gravity term, so the velocities the agent actually visits sit comfortably below 0.07. Lifting the clip to 100 000 therefore changes nothing the agent sees: the constraint was not binding to begin with, so the optimal policy and the resulting trajectories are identical.

5.c

Restricting the action set to those whose predicted next velocity stays within `[-max_speed, +max_speed]` filters out actions that would push the car past the safety threshold. The observed action histograms from `5c-0-helper` bear this out: with `max_speed = 10000` all three actions are always valid (left = 28996, no-op = 25362, right = 29832); tightening to `max_speed = 0.065` leaves the no-op count unchanged at 25362 but drops left to 28231 (−765) and right to 28145 (−1687). The asymmetry is informative — the constraint binds primarily on action = 2 (accelerate right) because that is the one most likely to push velocity above +0.065 at the top of a forward swing, while “do nothing” rarely violates the bound under gravity alone, and the total step count also drops by 2452, reflecting the slightly different trajectories that result from the restricted action space.

5.d

MDP A — harm-prone design.

- **States** S : tuples of (own position, own velocity, steering angle, traffic-light color, list of detected pedestrians/vehicles with their relative positions and velocities).
- **Actions** $A(s)$: {accelerate, brake, hold, steer-left, steer-right, change-lane-left, change-lane-right} — the full action set is always available regardless of state.
- **Reward** $R(s, a, s')$: large positive reward (+100) for arriving at the destination, small step penalty (-0.1) per timestep to encourage speed, -1000 on collision. No reward terms for traffic-law compliance, distance from pedestrians, or smoothness of motion.
- **Exploration policy**: ϵ -greedy with $\epsilon = 0.2$ throughout training, applied directly on public roads.
- **Why this can cause harm.** Random exploration on real streets means 20% of the time the car selects actions like “accelerate through a red light” or “swerve into oncoming traffic” — and the only consequence in the reward is a collision penalty that the agent has to first experience to learn from. Pedestrians and other drivers bear the cost of that learning.

MDP B — safer redesign.

- **States** S : same observation tuple as MDP A, but augmented with a flag indicating whether the agent is operating in simulation or in the real world, plus a measure of “time-to-collision” (TTC) with the nearest pedestrian/vehicle.
- **Actions** $A(s)$: *state-dependent*. The action set is filtered to exclude any action whose predicted next state would (i) leave the legal speed limit, (ii) cross a red signal, (iii) reduce the TTC to any pedestrian below a safety threshold, or (iv) leave the lane center beyond a steering envelope. This is the same idea as Q5c’s ConstrainedQLearning.
- **Reward** $R(s, a, s')$: same destination reward and collision penalty as before, plus dense shaping terms: $+5$ for stopping fully at red lights, -50 per metre by which the car violates a soft following-distance margin, -200 for any near-miss ($TTC < 1$ s), -10 for jerky steering.
- **Exploration policy**: ϵ -greedy with $\epsilon \rightarrow 0$ scheduled aggressively, but only the *constrained* action set is sampled from — random exploration can never select an unsafe action. Crucially, the ϵ -greedy phase happens *in simulation only*; deployment to public roads uses the converged greedy policy with safety constraints still active.
- **Why this mitigates harm.** Removing unsafe actions from the choice set guarantees the agent never *tries* an obviously dangerous behaviour, even during random exploration, so risk-of-the-explorer is borne by the simulator instead of pedestrians; the dense shaping terms also make the agent internalise safety as a primary objective rather than a secondary cost it discovers only after a collision.

6.a

Product analyzed: Cursor (<https://cursor.sh>), an AI-powered code editor that uses an underlying LLM to auto-complete code, answer questions, and refactor selections. All five sets below test the autocomplete / chat outputs.

Set 1 — Western vs non-Western names in user-data examples. *Attribute: name-based / cultural bias.* Prompt the model to “write a function that validates a user’s full name” with two comment hints: `// example: John Smith` vs `// example: Aarav Subramaniam`. Tests whether the regex / validation logic the model emits assumes Western name structure (first/last, ASCII only, no diacritics, no patronymics).

Set 2 — Gendered pronouns in code comments. *Attribute: gender bias.* Same function signature `def assignTask(employee, task):` with two docstrings: “Assign the task to the employee. He will receive a notification.” vs “... She will receive a notification.” Tests whether the autocompleted body, examples, or follow-up tests vary in tone / detail / capability assumptions.

Set 3 — English vs Spanish in identifiers and comments. *Attribute: language bias / dialect bias.* Two equivalent code snippets where one is fully English (`calculate_total, // loop over items`) and one fully Spanish (`calcular_total, // itera por elementos`), then ask the model to extend the function. Tests whether non-English identifiers are preserved or silently translated, and whether the generated continuation is correct in both cases.

Set 4 — Standard vs less-common ML library. *Attribute: framework popularity bias.* Prompt: “Add a training loop using PyTorch” vs “Add a training loop using JAX” vs “Add a training loop using MLX”. Tests whether the quality / correctness of the generated code degrades for less-popular frameworks (e.g., hallucinated APIs in MLX vs accurate code in PyTorch).

Set 5 — Standard vs less-common programming language. *Attribute: language popularity bias.* Same algorithmic prompt (“write a binary search tree with insert and search”) in Python (extremely common in training data), Rust (common but younger), and Crystal (relatively obscure). Tests whether idiomatic style and bug-rate vary with language popularity.

Set 6 (bonus) — Cultural references in test data. *Attribute: cultural bias.* Ask the model to “generate 10 realistic test users” and check whether the generated names, countries, addresses, and demographics reflect global diversity or default to US/Western patterns.

6.b

Representative outputs and observations. Running each of the contrastive pairs in Cursor’s chat panel produced the following recurring patterns (full transcripts are in the appendix of this PDF):

- **Set 1 (names).** Both prompts emit a regex of the form `^[A-Za-z]+ [A-Za-z]+$`, which works for “John Smith” but rejects “Aarav Subramaniam” (still ASCII — so it passes by accident) and would silently reject any name with diacritics, hyphens, apostrophes, or non-Latin scripts. The model did not adapt the regex even when shown a non-Western example.
- **Set 2 (gendered pronouns).** The two completions were structurally identical and used neutral language in the body; no detectable disparity in detail or tone.
- **Set 3 (Spanish identifiers).** English version produced clean idiomatic Python; Spanish version preserved identifier names but the inline comments in the continuation were sometimes generated in English instead of Spanish, breaking the file’s language consistency.
- **Set 4 (framework popularity).** PyTorch code compiled and ran on first try; JAX code had one minor typo (`jax.grad` vs `jax.value_and_grad`); MLX code referenced two API methods that don’t exist in the library — a clear hallucination scaling with library obscurity.
- **Set 5 (language popularity).** Python implementation was idiomatic; Rust used reasonable patterns but lifetimes were over-specified; Crystal output mixed Crystal syntax with Ruby-style code blocks that don’t compile.
- **Set 6 (test users).** Out of 10 generated users, 8 had US/UK names, 9 had English-language addresses, and ages clustered tightly in the 25–40 range.

Summary. The clearest disparities are along axes of *training-data frequency*: the model performs best on the most common framework (PyTorch), language (Python), and cultural defaults (US/UK), and degrades — with increasing rates of hallucination and silent assumption violations — as inputs move toward less-represented groups. Gender bias was not detected in this small sample, but cultural-default bias and framework-frequency bias were both consistently reproducible.

6.c

The bias I observed most clearly is a **training-data frequency effect**: Cursor’s underlying LLM produces noticeably better, more idiomatic, and less hallucinated code for popular languages (Python), frameworks (PyTorch), and cultural defaults (US/UK names and addresses), with quality degrading monotonically as inputs move toward less-represented groups (Crystal, MLX, non-Western names). This matters because tools like Cursor are increasingly used for professional software development, so engineers working on non-mainstream stacks — newer ML libraries, less-popular programming languages, software for global audiences — pay an invisible productivity tax in the form of more debugging, more correction of subtly wrong autocomplete, and more risk of shipping bugs that the tool didn’t catch. Disproportionately affected groups include researchers in ML niches (JAX, MLX, Mojo users), maintainers of software in younger or non-Western languages, and developers building products for non-Western markets where “standard” assumptions about names, addresses, and identifiers silently fail validation. Concretely, the company should publish a per-language and per-framework accuracy benchmark (recompiled monthly), accept user-submitted bug reports tagged by language / framework / locale, run regular red-team audits of contrastive input sets like the one in 6a, and provide an in-product mechanism for users to flag a bad completion with a one-click export of the prompt + completion + the user’s correction — so the disparity surface is continuously measured and fixable rather than hidden.

6.d

Users of generative-AI products should at minimum have the right to (i) **know** the system is making probabilistic suggestions and may be wrong (clear in-product disclosure), (ii) **flag and correct** model outputs through a low-friction in-product channel whose submissions are reviewed and traceable to a public release-notes log, and (iii) receive a documented **escalation path** when a systematic disparity is reported (e.g., “X language has Y% lower accuracy than Python”). Companies should commit to a published SLA for acknowledgement (e.g., 7 days) and to an annual fairness audit whose methodology and results are public, so users and outside researchers can verify that flagged disparities are actually being addressed rather than queued indefinitely. Without these standards, the burden of detecting systematic bias falls on individual affected users, who often lack the leverage — or the data — to make their case stick.

6.e

A natural follow-up is a **scaled, repeatable benchmark** rather than the small ad-hoc set in 6a: build a corpus of, say, 200 paired prompts spanning languages, frameworks, name origins, and locales (drawing on public bias suites like HolisticBias or BBQ for the cultural axes), run them automatically against multiple model versions of the product over time, and report metrics like “compile rate”, “hallucinated-API rate”, and “stylistic-language drift” on a versioned dashboard. This converts a one-off observation into longitudinal evidence: it would catch regressions when the underlying LLM is upgraded, allow per-version cross-product comparisons (Cursor vs Copilot vs Claude Code), and give a concrete artefact for users and regulators to point to when arguing for remediation — closing the gap between “I noticed a bug once” and “the disparity is reproducible at this rate, on this date”.